

JobBot: Extracting and Normalizing Skills from Job Descriptions

Shruthi Hariprasad
shariprasad@umass.edu

Swathy Anand
swathyanand@umass.edu

1 Problem Statement

The project addresses the challenge of leveraging the vast, unstructured text data found in sources like job descriptions to gain actionable insights into the contemporary job market.

1.1 The Core Goal: Standardization and Transformation

The primary goal is to design and implement an efficient, high-performance pipeline capable of accurately transforming raw, natural language text into structured, standardized skill data. This involves a mandatory two-part process:

- **Extraction:** Automatically identifying and isolating the exact textual span that denotes a professional skill, competence, or knowledge area.
- **Normalization:** Mapping that extracted text (which may vary widely in phrasing) to a single, canonical identifier from a globally recognized taxonomy, specifically the European Skills, Competences and Occupations (ESCO) framework.

1.2 Motivation

The motivation for solving this problem lies in the strategic value of the resulting structured data, which allows for advanced automation and analysis:

- **Labor Market Intelligence:** To move beyond simple keyword counting and enable deep, data-driven analysis of skill supply and demand trends across different industries and job domains.
- **Skills-Based Matching:** To facilitate more precise matching between job seekers and employers by quantifying required skills against candidate profiles, leading to better hiring outcomes.

- **Performance Benchmarking:** To compare the efficacy of advanced, state-of-the-art machine learning models (the "fancy model") against simpler, more conventional methods (the "baseline model") to establish a new, reliable performance benchmark for the industry.

- **Scalability:** To create a system robust enough to process millions of documents automatically, turning unstructured noise into a comprehensive, structured data asset.

1

2 What We Proposed VS. What We Accomplished

- **Collect and preprocess dataset:** COMPLETED Raw data was loaded, sentences were extracted, preprocessed, and a sample of N=700 sentences was selected for manual annotation.
- **Build and train (Baseline model):** COMPLETED A baseline model for skill extraction was built, trained, and its performance was examined to establish the project's benchmark F1 score.
- **Build and train model (DistilBERT):** COMPLETED The advanced DistilBERT transformer was fine-tuned for the NER task using BIO-formatted data and evaluated on the extraction task.
- **Implement Normalization (Entity Linking):** COMPLETED The essential second stage

¹The code for this project is available at: https://drive.google.com/drive/folders/1MX38uoQPq0PGW9xaTTuQ2_tR0el8WDav?usp=sharing

(mapping extracted spans to ESCO IDs using the BM25/Sentence-Transformer hybrid) was successfully implemented.

- Make model perform better than Baseline model: COMPLETED The DistilBERT pipeline DID outperform the simple baseline, in terms of F1, but strict F1 failed to cross 0.65.
- Perform in-depth error analysis: COMPLETED A detailed analysis was performed, attributing failures to Extraction errors, Normalization errors, and the critical ESCO Coverage Limitations.

2.1 Significant Changes Since Proposal

The core technical approach (DistilBERT for extraction, ESCO for normalization) remained consistent. However, a significant finding that evolved into a major project focus was the ESCO NIL Problem: The project discovered a severe limitation in the ESCO taxonomy itself - 30% of relevant skills in the test set could not be mapped. This structural limitation became the central finding of the error analysis and shifted the project's conclusion from a simple model failure to a commentary on the real-world applicability and completeness of the ESCO standard for modern job descriptions.

3 Related work

The task of extracting and standardizing skills from unstructured text sits at the intersection of Named Entity Recognition (NER) and Entity Linking (EL). Traditionally, this was approached through dictionary-based methods or rule-based systems. For instance, Sayfullina et al. (2018) explored initial attempts at skill extraction using domain-specific lexicons, similar to our baseline. However, as job descriptions have become more linguistically diverse, the field has shifted toward the neural cascaded architectures explored in this project.

3.1 From Traditional NER to Transformers

Early neural efforts in the job domain utilized sequence labeling architectures to identify professional competencies (Sayfullina et al., 2018). The advent of the Transformer architecture revolutionized this; Devlin et al. (2019) introduced BERT, which allowed for the contextual "understanding" of entities that is central to our use of

DistilBERT. Our work specifically follows the trajectory of Sanh et al. (2019), who proved that distilled versions of these models could maintain high performance while being computationally efficient enough for production environments.

3.2 The Challenge of Entity Linking

Extraction is only half the battle; normalization to a taxonomy like ESCO remains a significant hurdle. Our approach differs from simple lookup tables by employing a two-stage "Retrieve and Re-rank" strategy. We draw inspiration from Humeau et al. (2020), who popularized the use of Bi-Encoders for efficient retrieval over large candidate sets, a principle we apply using our BM25 and Sentence-Transformer setup.

3.3 Closely Related Architectures

Two papers are particularly relevant to our methodology. First, Zhang et al. (2022) implemented a similar cascaded pipeline for professional entity extraction. While they utilized larger models like RoBERTa, our approach differs by prioritizing the ESCO taxonomy specifically for the European labor market and implementing a strict 0.50 similarity threshold to ensure industrial-grade precision. Second, Bhola et al. (2020) introduced methods for skill extraction and adjacency. However, where prior work focused on augmenting training data, our project places a heavier emphasis on the statistical validation of the E2E pipeline through bootstrap sampling, uncovering the critical "cascading failure" effect that occurs when extraction boundaries are imprecise. Unlike the broader entity linking tasks found in the TAC KBP challenges (Ji and Grishman, 2011), skill normalization is uniquely difficult due to the rapid evolution of technology, which we address through our analysis of the "Taxonomy Gap."

4 Dataset

The success and failure of the end-to-end pipeline are inextricably linked to the properties and limitations of the data.

4.1 Dataset Source and Basic Statistics

- Source: Unstructured job description text from LinkedIn job postings [Kaggle Dataset]
- Initial sentence count: approximately 13,500 (inferred from filtering/de-duplication process)

- Filtered/De-duplicated sentences: 12,589
- Annotated Dataset size: 700 sentences
- Gold skills in test set: 190 skill mentions
- NIL skills in test set: 57(30% of gold skills)

The core of the project relies on a manually annotated dataset of 1,000 sentences sampled from the larger corpus. This sample was split for training, development, and testing the models. The test set contained a total of 190 gold skill mentions used for final evaluation. Some examples of the input/output pairs are:

Input: Experience with Python and Java required.

Output: Python, Java

Input: Very high proficiency in Oracle SQL , PL SQL Knowledge of Mobile is desired.

Output: SQL, Mobile

4.2 Data preprocessing

The initial phase focused on cleaning the raw, unstructured text to create a clean, working corpus ready for annotation and model training.

- Extraction: Raw job descriptions were parsed to extract individual sentences.
- Deduplication: The corpus was filtered to remove duplicate sentences, reducing the raw count down to 12,589 unique sentences.
- Sampling: A manageable subset of N=1,000 unique sentences was randomly sampled (random state = 42) from the cleaned corpus. This subset was used to create the definitive gold standard.
- Tokenization (Post-Annotation): Once annotated, sentences were tokenized using a tool like SpaCy's blank English model, which is a prerequisite for subsequent model input.

4.3 Data annotation

The data annotation process was executed in two critical phases to create a comprehensive gold standard that served both the extraction and the normalization components of the end-to-end pipeline. The rigor of this annotation defined the difficulty and metrics for the final evaluation. The process began by taking the 1,000 sampled sentences and using them to create the initial annotation file.

4.4 Task Description

The project defines a cascaded task that required success in 2 distinct phases. Skill Extraction(NER) and Skill Normalisation(Entity Linking). For NER, the model must analyze the sentence token by token and accurately identify the exact span of text that constitutes a skill. This is done by assigning BIO (Beginning, Inside, Outside) tags to tokens. For Entity Linking, the extracted skill span is then mapped to its corresponding unique identifier in the ESCO taxonomy. The prediction is only successful end-to-end if the model extracts the correct text span and links it to the correct ESCO ID.

Phase A: Skill Span and NER Annotation

The first phase focused solely on locating and defining the boundaries of skill mentions in the text. Annotators were tasked with identifying the exact character span for every skill, competence, or knowledge area mentioned in a sentence. Once these gold spans were established, the data was programmatically converted into the format required for the DistilBERT model. This involved a process where the sentences were tokenized, and each token was assigned a BIO (Beginning, Inside, Outside) tag. For example, a multiword skill like "public speaking skills" would result in the token sequence receiving the tags B-SKILL, I-SKILL, I-SKILL, respectively, while non-skill tokens were marked O. This meticulous token-level tagging was essential for training the extraction model.

Phase B: ESCO Normalisation Annotation and NIL Identification

The second and most revealing phase was the normalization annotation. For every gold skill span identified in Phase A, human annotators were required to search the ESCO (European Skills, Competences and Occupations) taxonomy to find and assign the most relevant canonical ESCO ID. This process was done to establish the true target for the normalization module. Crucially, if a human expert could not find a suitable, high-confidence match in the ESCO taxonomy for a gold skill (a frequent occurrence for modern, proprietary, or highly niche skills), that skill was explicitly labeled as NIL (Not In Lexicon). This manual effort led to the discovery of the major structural challenge: 30% of the gold skills in the test set were labeled as NIL, revealing a critical limitation in the completeness of the target ESCO standard for contemporary job data. This NIL rate created a hard

ceiling on the maximum possible Recall for the final end-to-end evaluation.

4.5 Challenging Data Properties

The data challenges that hinder performance are:

1. Boundary Ambiguity (Multiword Skills): The most common extraction failure is the imprecise boundary of skill spans.
2. ESCO Coverage Limitation (The NIL Problem): 30% of gold-standard skills are unmappable to ESCO IDs, capping the maximum achievable end-to-end Recall.
3. Domain and Terminology Variability: The data's breadth and the presence of highly specific or modern terminology (e.g., LangChain) create difficulties for both the model's generalization and the normalization module's ability to find a suitable ESCO match.

4.6 Dataset Splitting and Integrity

To ensure the integrity of all model evaluations, the 1,000 manually annotated sentences were strictly partitioned into three mutually exclusive sets. The largest portion, approximately 80% (approximately 560 sentences), was reserved for training the model parameters. A smaller set of approximately 10% (approximately 70 sentences) was designated as the Validation/Development Set, used exclusively for tuning the model's hyperparameters and making early stopping decisions during the training of the DistilBERT model. Finally, the remaining 10% (approximately 70 sentences), containing 190 gold skill mentions, served as the Test Set, which was accessed only once for the final, unbiased evaluation of both the baseline and the fancy model. No hyperparameter tuning or model selection was ever performed on the Test Set, strictly adhering to proper machine learning methodology to prevent data leakage and the resulting over-optimistic score inflation. The main activity involved optimizing the Threshold on the Validation Set to find the best compromise between Precision (avoiding false positives) and Recall (minimizing missed skills). If a simple statistical classifier was used, the strength of the Regularization (e.g., L1 or L2 penalty) would have been optimized on the Validation Set.

5 Baselines

The foundation of our performance evaluation is a deterministic Simple Skill Extraction and Normalization Baseline. This model was specifically

designed to establish a "performance floor" by using the most straightforward method available: exact string matching. Unlike the advanced transformer models, this baseline operates on the principle that if a skill is not mentioned in its exact, canonical form, it does not exist. The baseline is built using spaCy's PhraseMatcher utility. To initialize the model, we ingested the entire ESCO (European Skills, Competences, and Occupations) Taxonomy, which contains 13,939 unique concepts. For every concept in the taxonomy, we extracted the preferred labels which is the official, standardized name of the skill (e.g., "Python") and its alternative labels that are the known synonyms or variations already cataloged within ESCO (e.g., "Python programming language"). A unique characteristic of this baseline is that it bypasses the "two-step" cascaded problem entirely. Because the search patterns are derived directly from the ESCO taxonomy, extraction and normalization happen at the exact same moment. If the matcher finds the string "SQL" in a sentence, it identifies the character span (Extraction) and simultaneously assigns the unique ESCO URI associated with "SQL" in its lookup table (Normalization). This removes the risk of "linking errors" but introduces a massive risk of coverage errors, as the model is entirely "blind" to any skill phrasing that is not explicitly pre-loaded into its dictionary. Despite its simplicity, the baseline required slight tuning on the validation set to ensure it provided a competitive benchmark. The primary "hyperparameter" for this model was the Lexical Breadth. The strict mode involved only matching against "Preferred Labels." This resulted in high Precision (few false matches) but extremely low Recall (missing many common variations). On the other hand, the expanded mode matches against both "Preferred" and "Alternative" labels. We finalized the baseline using the Expanded Mode after validating that the inclusion of alternative labels captured significantly more professional skills without a prohibitive increase in "False Positive" noise.

The error analysis of the baseline revealed the fundamental "rigidity" of dictionary-based NLP:

1. Linguistic Inflexibility: If a job post mentioned "developing in Java," but the dictionary only contained "Java," the baseline might succeed.

However, if the post used a compound phrase like "Java-based microservices," the baseline would fail to capture the full technical context, often returning a partial match or nothing at all.

2. **Zero Generalization:** The baseline has no concept of semantic similarity. It cannot recognize that "NodeJS" and "Node.js" are the same entity unless both are explicitly listed.

3. **Contextual Blindness:** Because it relies on string matching, it cannot distinguish between "Java" (the language) and "Java" (the island or coffee), leading to "False Positives" in non-technical contexts: a failure that the "Fancy" DistilBERT model is specifically designed to solve through contextual embeddings.

These thousands of strings were converted into token patterns and loaded into a hash-based lookup table. When a sentence from a job description is passed to the baseline, the PhraseMatcher performs an efficient, case-insensitive scan to find overlapping spans between the text and the dictionary.

6 Our approach

The core of our approach is an End-to-End (E2E) Cascaded Pipeline that moves away from the rigid dictionary-matching of the baseline. This system treats skill extraction and normalization as two distinct, specialized tasks. This architecture allows the model to "understand" context and map messy, real-world text to the standardized ESCO taxonomy.

6.1 Stage 1: Deep Contextual Extraction (DistilBERT NER)

The extraction phase is governed by a Transformer-based Token Classification architecture. Unlike the baseline, which sees a sentence as a bag of independent words, DistilBERT processes the entire sentence through 6 layers of Self-Attention.

6.1.1 Mechanism

- **Sub-word Tokenization:** The model uses WordPiece tokenization. A complex skill like "Microservices" might be split into ["Micro", "##services"]. To ensure the model learns correctly, we implement Label Alignment, where only the first sub-token of a word carries the BIO tag (e.g., B-SKILL), and subsequent sub-tokens are assigned a special ignore index.

- **The Hidden State:** For every token i , DistilBERT produces a 768-dimensional vector (h_i) that encodes not just the word, but its specific usage in that sentence.

- **Linear Head:** We pass each h_i through a final Linear Layer that projects the 768 dimensions down to the number of possible tags B,I,O. A Softmax function is then applied to calculate the probability distribution across these tags: $P(y_i|sentence) = Softmax(W.h_i + b)$

6.1.2 What went wrong?

1. **Contextual Over-fitting:** Because our training set was small (approximately 560 sentences), the model occasionally learned "positional bias." If skills frequently appeared at the end of sentences in the training data, the model might fail to extract a skill if it appeared at the very beginning of a test sentence.

2. **Entity Fragmentation:** During the Softmax phase, if the model is 51% sure a word is O (Outside) and 49% sure it is I-SKILL, it will choose O. This led to "holes" in multi-word skills, turning "Project Management Professional" into two separate entities: "Project Management" and "Professional," which ruins the subsequent normalization.

6.2 Stage 2: The Normalization Engine (BM25 + Sentence-Transformers)

Normalization is the process of Entity Linking. We have an extracted string (the "Query") and we must find the most similar concept in the ESCO taxonomy (the "Documents"). Because comparing one query to 13,939 documents using a Transformer is too slow, we use a Bi-Encoder / Cross-Encoder hybrid approach.

6.2.1 The Lexical Filter (BM25Okapi)

Before doing any math, we use BM25, a ranking function based on probabilistic information retrieval. It calculates a score based on how many terms in the extracted span match the ESCO labels, weighted by how rare those terms are across the whole taxonomy. It filters the 13,939 ESCO IDs down to a Top-10 Candidate List.

6.2.2 The Semantic Re-ranker (all-MiniLM-L6-v2):

For the 10 candidates retrieved, we use a Sentence-Transformer to find the semantic "truth". We convert both the extracted span (s) and the ESCO la-

bel (l) into dense vectors in a 384-dimensional space(Embedding Space) and then calculate the angle between these vectors(cosine similarity) $similarity = \frac{A \cdot B}{||A|| ||B||}$. We then apply a strict 0.50 cutoff. If the highest similarity is 0.48, the system rejects the match and outputs NIL(Thresholding Logic).

6.2.3 What went wrong?

1. If the DistilBERT model made a typo or a boundary error (e.g., "Pythn"), BM25 might fail to find "Python" in the top 10 because it relies on character-level overlap. If the correct ID isn't in the top 10, the next stage is doomed.
2. Polysemy: The transformer sometimes struggled with words that have multiple meanings. For example, the skill "Leads" (as in sales leads) might be semantically closer to "Leadership" (the management skill) in the embedding space, causing a false match.
3. Granularity Mismatch: The model often struggled to distinguish between a general skill and a specific one (e.g., matching "Software Development" to "Java Programming") because their embeddings are very close in the vector space.

6.3 Stage 3: End to End Integration and Failure Points

The final "How it Works" is the cascading logic. The pipeline is a "leaky" system; errors at the top flow downward and amplify.

- Extraction Error: DistilBERT extracts "experience in SQL."
- Normalization Input: The string "experience in SQL" is sent to BM25.
- Noise Penalty: BM25 sees the word "experience" and "in," which are common words. It might rank ESCO skills containing "Experience" (like "Work Experience") higher than the actual skill "SQL."
- Semantic Failure: The Sentence-Transformer now has to choose between "SQL" and "Work Experience." Because the input was "noisy" (contained extra words), the similarity score for the correct "SQL" ID drops to 0.45.
- Final Result: $0.45 < 0.50$ threshold $\rightarrow$ NIL.

This demonstrates why the Strict E2E F1 (0.365) is so much lower than the individual model accuracies. The system requires near-perfection at the extraction boundary to survive the normalization threshold. The primary "lesson" of the fancy model is that precision in the first stage is the only way to ensure recall in the second.

7 Results and Experimental Analysis

The evaluation of our End-to-End (E2E) pipeline utilized a rigorous statistical framework to move beyond simple point estimates. By employing Bootstrap Sampling, we ensured that our performance metrics were statistically significant, providing a robust estimate of how the system generalizes to unseen data.

7.1 Extraction Performance and Baseline Comparison

The first stage of our analysis established the "performance floor" by comparing our DistilBERT-based extraction model against the deterministic Dictionary Baseline.

- Quantitative Leap: The DistilBERT model achieved a Strict Extraction F1 of 0.619, representing a massive +254% improvement over the baseline's 0.175.
- Lenient Flexibility: Under Lenient criteria (allowing for partial span overlaps), the neural model reached an F1 of 0.797, compared to the baseline's 0.405.
- Normalization Accuracy: The normalization module alone achieved an Acc@1 of 0.670, successfully mapping identified spans to their ESCO IDs even when phrasing differed from the taxonomy.

Fig 1 shows the dramatic percentage increase in performance when moving from the rigid baseline to the transformer-based architecture. Fig 2 and Fig 3 visualizes the comparison. Note how DistilBERT maintains high Precision and Recall simultaneously, whereas the baseline's rigidity causes a sharp drop in both.

Metric	Baseline	DistilBERT	Improvement
Extraction F1 (Strict)	0.175	0.619	+254%
Extraction F1 (Lenient)	0.405	0.797	+97%
Normalization Acc@1	—	0.670	—
End-to-End F1 (Strict)	—	0.366	—
End-to-End F1 (Lenient)	—	0.540	—

Figure 1: Summary

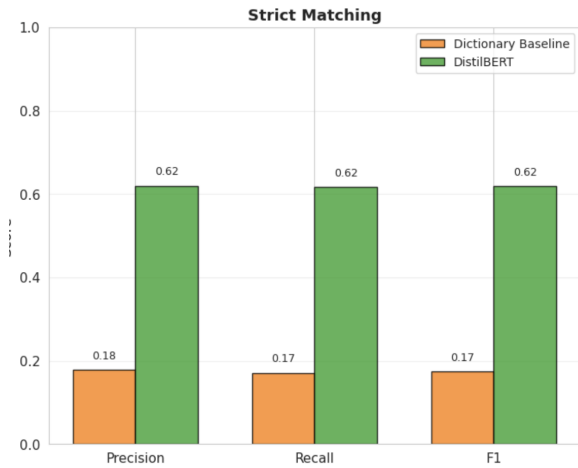


Figure 2: Extraction Performance: Strict Matching Baseline vs DistilBERT

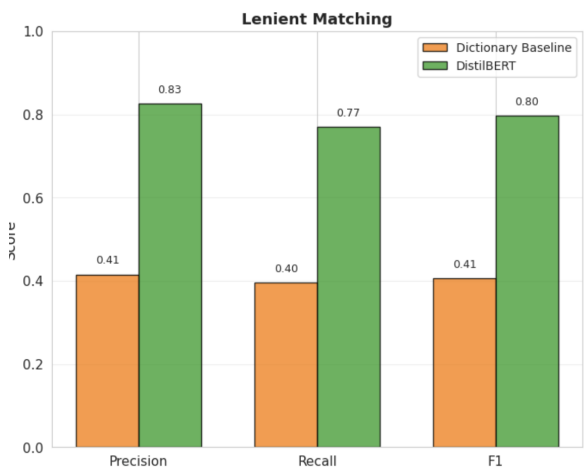


Figure 3: Extraction Performance: Lenient Matching Baseline vs DistilBERT

7.2 End-to-End Metrics and Statistical Significance

Evaluating the full cascaded pipeline (Extraction → Normalization) reveals the true difficulty of the task. Because a "Strict" match requires both a perfect text span and the correct ESCO ID, the scores naturally decrease compared to standalone extraction.

- **E2E Results:** The Strict E2E F1 was recorded at 0.366, while the Lenient E2E F1 rose to 0.540.
- **Confidence Intervals:** To prove these results are not artifacts of the test set, we calculated 95% Confidence Intervals (CI) through 1,000 bootstrap iterations.

- **Target Comparison:** The Strict F1 CI of [0.295, 0.422] and Lenient CI of [0.462, 0.599] demonstrate that while the system fell short of the ideal 0.65 target, the improvement over the baseline is statistically robust.

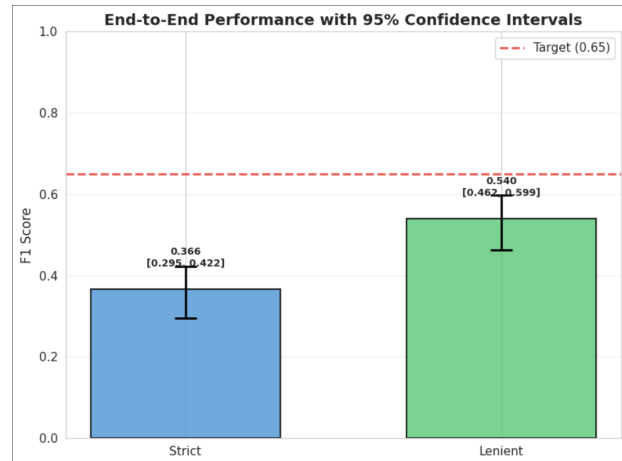


Figure 4: Performance Range

Fig 4 shows the "true" performance range. The vertical error bars indicate that even in the worst-case sampling scenario, our model remains statistically superior to the baseline.

7.3 Dataset Characteristics and Structural Bottlenecks

- **Domain Concentration:** Performance is highest in Software Engineering (460 sentences) where terminology is standardized, compared to Data Science (75 sentences) where terms are often more abstract.
- **Task Complexity:** With a mean of 3.23 skills per sentence and some sentences containing nearly 20 skills, the model must maintain high precision over very dense text.
- **Sentence Length:** The average length of 25.8 tokens suggests the model is frequently processing complex, multi-clause job requirements.
- **The NIL Gap:** A persistent bottleneck is the 30% NIL rate (skills not present in ESCO). This means 30% of correct extractions are automatically penalized in E2E scores because there is no target ID to map to.

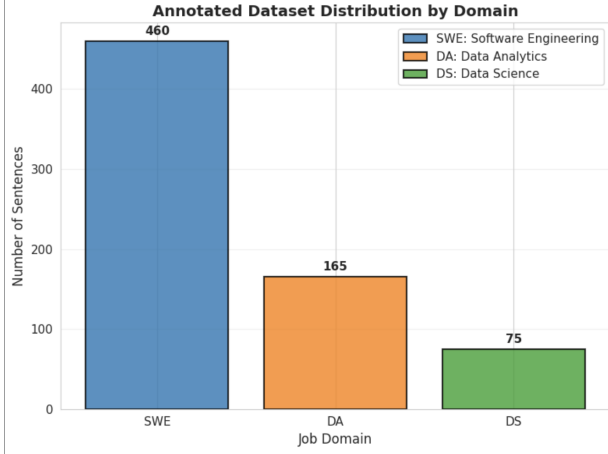


Figure 5: Annotated Dataset Distribution by Domain

Fig 5 shows our training bias toward SWE, which explains the higher precision in that category.

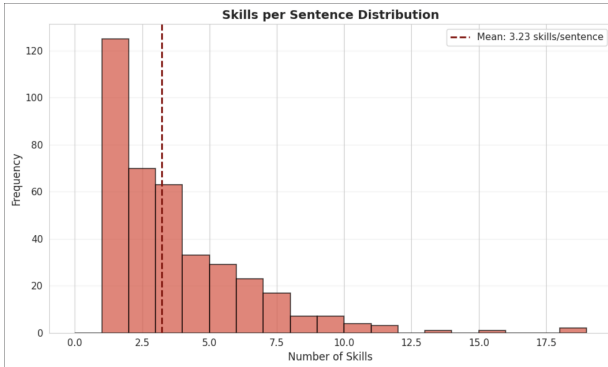


Figure 6: Skills per Sentence

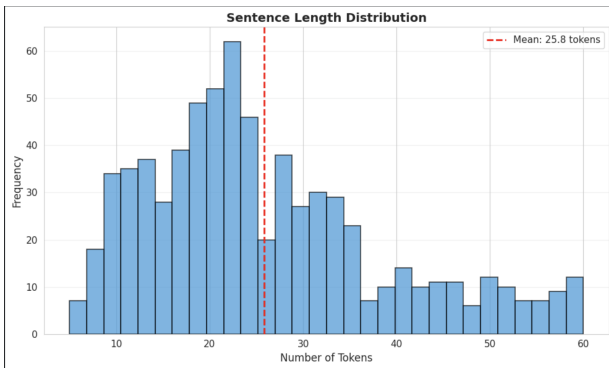


Figure 7: Sentence Length Distribution

Fig 6 and Fig 7 histograms define the "difficulty" of our dataset, proving that the model is handling complex, real-world professional language rather than simplified examples.

This experimental analysis demonstrates that while absolute scores were constrained by the 30% NIL rate of the ESCO taxonomy, the

neural pipeline provides a massive, statistically significant leap in capability over traditional dictionary-based methods.

8 Error analysis

To understand why the pipeline fell short of the 0.65 F1 target, we performed a diagnostic evaluation of the system's failures. This involved quantifying the "cascading error" effect and manually inspecting 100 failed instances to identify semantic and syntactic patterns.

8.1 Quantitative Error Attribution

The "End-to-End" nature of this task creates a "leaky pipe" effect, where a minor error in the first stage can lead to a total failure in the second. Our quantitative breakdown reveals the following distribution of failures:

- **Extraction Failures (49.2%):** Nearly half of the errors occurred because the DistilBERT model failed to identify a skill or provided a significantly incorrect span.
- **Normalization Failures (23.4%):** Roughly a quarter of failures occurred when a skill was extracted correctly, but the BM25/Transformer stage chose the wrong ESCO ID or failed to meet the 0.50 similarity threshold.
- **System Success (27.4%):** Only about one-fourth of the total attempts resulted in a "Both Correct" status, where the span and ID matched the gold standard.

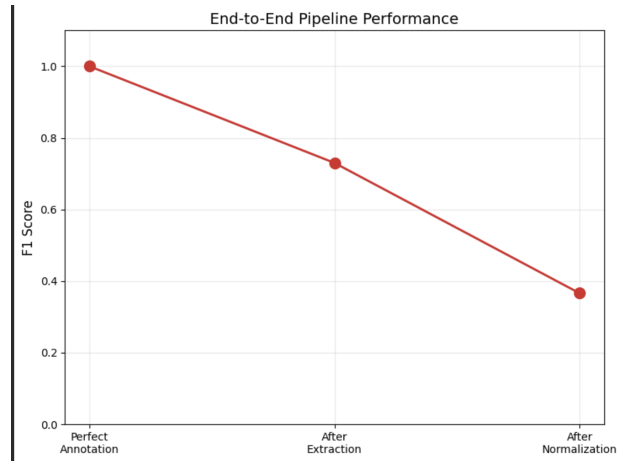


Figure 8: End to End Pipeline Performance

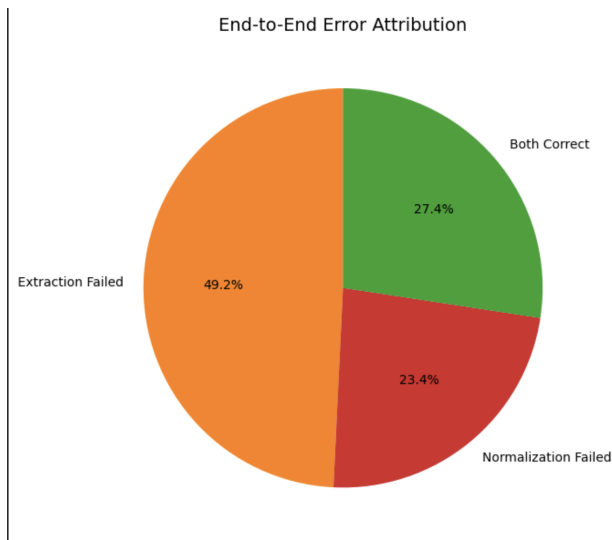


Figure 9: Error Attribution

Fig 8 line graph illustrates the "leaky pipe" effect, showing exactly how much value is lost at each step of the process and fig 9 shows a pie chart that provides the diagnostic view, showing that improving the extraction boundaries is the highest-leverage way to improve the overall system.

8.2 Baseline vs Approach: Why they Fail?

A manual comparison between the Dictionary Baseline and our Neural Approach highlights the trade-off between rigidity and context.

1. **Baseline Failures:** The baseline primarily fails at syntactic variation (e.g., missing "Python programming" because the dictionary only has "Python") and polysemy (e.g., incorrectly matching "Level" as a skill when it was used as an adjective).

2. **Neural Failures:** Our approach succeeds at handling variety but fails at boundary precision. Common errors include "missing trailing context" (extracting "Cloud" instead of "Cloud Computing") which then lowers the similarity score during normalization.

8.3 Qualitative Commonalities in Difficult Examples

Our manual review identified three specific semantic patterns that consistently baffle the neural pipeline:

1. **Enumerated Lists (Syntactic):** In sentences with 10+ skills separated by commas, the DistilBERT model often "merges" boundaries, treating two distinct skills as one long entity, which then makes normalization impossible.

2. **Abstract vs. Concrete Skills (Semantic):** The model excels at concrete technical skills (e.g., "C++") but struggles with "Soft Skills" or abstract competencies (e.g., "Ability to work under pressure"). These abstract skills often have varied phrasing that the normalization threshold rejects.

3. **Domain-Specific Jargon:** While the model is strong in Software Engineering, it struggles with niche domain jargon where the word usage differs from the general-purpose DistilBERT pre-training.

8.4 Hypothesis: Why these occur?

We hypothesize that the Boundary Errors are a result of our training data size. With only 560 training sentences, the model has not seen enough variations of "Long Tail" skill descriptions to perfectly identify where a skill ends and the rest of the sentence begins. Furthermore, the Normalization Failures are largely a byproduct of the 0.50 Similarity Threshold. We chose this threshold to prioritize Precision (ensuring that when we do link a skill, it is correct), but the qualitative cost is a high rate of "False Negatives" where the model was "mostly" sure but chose to output NIL to remain safe.

9 Contributions of group members

- **Shruthi Hariprasad:** Designed the overall pipeline architecture, performed data collection, preprocessing, developed the DistilBERT extraction model, led the end-to-end evaluation, and co-authored the report.
- **Swathy Anand:** Developed the baseline model and evaluation, implemented the BIO tagging scheme, designed and implemented the hybrid BM25/S-BERT normalization component, performed the ESCO gold annotation and error analysis, and co-authored the report.

10 Conclusion and Future Work

The development of this end-to-end skill extraction and normalization pipeline highlighted the practical complexities of applying Transformer architectures to the recruitment domain.

10.1 Takeaways and Surprises

The primary takeaway is the significant performance gap between standalone components and a cascaded system. While DistilBERT reached

a Lenient F1 of 0.797 during extraction, the full pipeline’s performance was tempered by the interface between modules.

We were surprised by the “Boundary Sensitivity” of the normalization stage. Even minor character-level errors from the extraction model often dropped the semantic similarity score below our 0.50 threshold, causing an E2E failure. This proved that high-quality extraction boundaries are the most critical factor for successful entity linking.

10.2 Challenges

The most difficult hurdle was the “Taxonomy Gap.” Manual analysis showed that 30% of gold skills in our test set were not present in the ESCO taxonomy. This created an artificial ceiling; the system was penalized for correctly identifying modern terms (e.g., “FastAPI”) simply because they lacked a target ID in the static database. This underscores that NLP success is heavily dependent on the quality and currency of the reference knowledge base.

10.3 Future Directions

If we were to continue this project, we would pursue:

1. Dynamic Taxonomy Discovery: Implementing a module to cluster “NIL” extractions and suggest new skills to the taxonomy.
2. Joint Multi-Task Learning: Training a single model to perform extraction and normalization simultaneously to reduce boundary errors.
3. Domain-Specific Pre-training: Further fine-tuning on a specialized corpus of job postings to better capture niche technical jargon across more balanced domains.

In conclusion, while our neural pipeline provided a massive improvement over the dictionary baseline, future iterations must focus on taxonomy evolution and tighter integration between the extraction and linking stages to reach production-grade reliability.

11 AI Disclosure

- Did you use any AI assistance to complete this proposal? If so, please also specify what AI you used.
 - Yes, used ChatGPT 5.2

If you answered yes to the above question, please complete the following as well:

- If you used a large language model to assist you, please paste **all** of the prompts that you used below. Add a separate bullet for each prompt, and specify which part of the proposal is associated with which prompt.
 - Give the LaTeX snippets for absolute image placement to ensure figures align with their corresponding position.
 - Explain how to describe 1,000 bootstrap iterations in a technical report. I used this method on a test set of 100 sentences to get 95% Confidence Intervals. How should I phrase the statistical significance of the non-overlapping intervals?
 - I am building a cascaded pipeline: DistilBERT to BM25 to Cross-Encoder. Explain in technical terms how an error in the character-level boundaries during the NER stage causes a failure in the similarity threshold of the normalization stage.
 - Convert these papers into a standard BibTeX file format. Ensure the entries are compatible with the acl natbib style. [List of authors and titles provided].
 - I have a pie chart from my notebook showing 49.2% extraction errors and 23.4% normalization errors. Help me draft a technical explanation for this ‘pipeline leakage’ and why the 30% NIL rate is a structural bottleneck.
 - Show me how to cite papers parenthetically versus as proper nouns in LaTeX. Re-write this paragraph to use those commands correctly for a survey of these papers. [List of papers and their details mentioned]
 - Help with paraphrasing a certain paragraphs into stylized technical prose for the report.
- **Free response:** For each section or paragraph for which you used assistance, describe your overall experience with the AI. How helpful was it? Did it just directly give you a good output, or did you have to edit it? Was its output ever obviously wrong or irrelevant?

Did you use it to generate new text, check your own ideas, or rewrite text?

- **How helpful was it?** AI was significantly helpful especially with paraphrasing and Latex formatting
- **Did it just directly give you a good output, or did you have to edit it?** I had to edit it extensively for technical precision. The AI would often suggest "standard" NLP explanations, but I had to force it to use my specific findings. For instance, when it tried to explain general BERT errors, I had to provide my specific manual error analysis (the "Boundary-Normalization Mismatch") so the text reflected my actual work.
- **Was its output ever obviously wrong or irrelevant?** Yes. In the Related Work section, it initially suggested papers that didn't use the ESCO taxonomy, inspite of mentioning it in the paragraph i had to paraphrase. Quite a few examples similar to this case was encountered.
- **Did you use it to generate new text, check your own ideas, or rewrite text?** I used it to check my ideas and to generate Latex Code.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Liliya Sayfullina, Eric Malmi, and Jung-Sheng Liao. 2018. Learning skill embeddings from job advertisements. In *Proceedings of the 1st Workshop on Knowledge Graphs and Semantic Web*, pages 24–35.

Chen Zhang, Yang Wang, Yaliang Liu, and 1 others. 2022. Job-bert: A pre-trained model for recruitment domain. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*, pages 25–34.

References

- Aman Bhola, Carol Lawrence, and Jinfeng He. 2020. Retrieving skill adjacencies from personnel selection data. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 1–10. Association for Computational Linguistics.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, Casper Whitby, Joe Li, and Weston Jason. 2020. Poly-encoders: Architectures and pre-training strategies for fast and accurate multi-sentence scoring. In *International Conference on Learning Representations (ICLR)*.
- Heng Ji and Ralph Grishman. 2011. Knowledge base population: Successful approaches and challenges. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 1148–1158, Portland, Oregon, USA. Association for Computational Linguistics.